

Bachelorthesis
Attribute Stream-Based Access Control im
Linux-Kernel

Christian Koch

11. Juni 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation	4
1.2	Wissenschaftliche Fragestellung	4
1.3	Zielsetzung	4
1.4	Aufbau der Arbeit	4
2	Grundlagen	5
2.1	Zugriffskontrolle und Reference-Monitor-Konzept	5
2.2	Attribute-Based Access Control	5
2.3	Attribute Stream-Based Access Control	5
2.4	Policy-Beschreibungen für ASBAC	5
2.5	Userspace und Kernelspace in Linux	5
2.6	Linux Security Modules und LSM-Hooks	5
2.7	eBPF und eBPF-LSM	5
2.8	eBPF-Maps und gepinnte Maps	5
2.9	eBPF-Verifier, Stack und Programmkomplexität	5
2.10	Aya als Rust-Framework für eBPF	5
2.11	Dynamische Attribute und Policy-Generationen	5
2.12	Überwachung bestehender Zugriffe und File Descriptors	5
3	Anforderungsanalyse	6
3.1	Funktionale Anforderungen	6
3.2	Operative Anforderungen	7
3.3	Entwicklungsbezogene Anforderungen	8
4	Konzeption	9
4.1	praktische Einschränkungen von eBPF	9
4.2	Architektur	9
4.3	Datenfluss	9
4.3.1	Policy-Datenfluss	10
4.3.2	Datenfluss dynamischer Attribute	10
4.3.3	Zugriffsentscheidungen im Kernel	11
4.3.4	Monitoring bestehender Zugriffe	11
4.4	Komponenten	11
4.4.1	Userspace-Komponenten	11
4.4.2	Kernelspace-Komponenten	13
4.4.3	Administrationskomponente	13
4.5	Datenobjekte	13
4.5.1	Datenobjekte im Userspace	13
4.5.2	Datenobjekte im Kernelspace	13
4.6	Entwurfsentscheidungen	14
4.6.1	Trennung von Userspace und Kernelspace	14
4.6.2	Einsatz von eBPF-LSM statt klassischem Kernelmodul	14
4.6.3	Auswahl der LSM-Hooks	15
4.6.4	Policy-Verarbeitung im Userspace	15

4.6.5	Reduzierte Policy-Repräsentation im Kernelspace	15
4.6.6	Kommunikation über eBPF-Maps	15
4.6.7	Generationenmodell für Policy-Aktualisierungen	15
4.6.8	Trennung statischer Policies und Streampolicies	15
4.6.9	Aufteilung der Policy-Maps nach Hook und Policy-Typ	15
4.6.10	Tail-Call-basierte Aufteilung der eBPF-Programme	15
4.6.11	Combining-Strategie für Policy-Entscheidungen	15
4.6.12	Identifikation von Dateien über Device und Inode	15
4.6.13	Nachträgliche Überwachung durch den Monitor	15
4.6.14	Enforcement durch Schließen von File Descriptors	15
4.6.15	Administration und Diagnose über ein separates Tool	15
5	Entwurf und Umsetzung	16
5.1	Komponente 1	16
5.2	Komponente 2	16
5.3	Komponente 3	16
5.4	Implementierungsdetails	16
6	Evaluation	17
6.1	Testumgebung	17
6.2	Testszenarien	17
6.3	Ergebnisse	17
7	Zusammenfassung und Ausblick	18

1 Einleitung

1.1 Motivation

Die Sicherheit von Linux-Systemen ist ein fundamentales Anliegen in der heutigen IT-Landschaft. Das Linux Security Module Framework stellt Mechanismen für verschiedene Sicherheitsprüfungen mittels Attribute-Based Access Control im Linux-Kernel dar[3]. LSM-Hooks ermöglichen es, sicherheitsrelevante Kerneloperationen abzufangen und auf dieser Grundlage Zugriffsentscheidungen zu treffen. Ein LSM-Hook prüft einen Zugriff jedoch grundsätzlich zu dem Zeitpunkt, an dem die jeweilige Kerneloperation ausgeführt wird. Bei dauerhaft bestehenden Zugriffen wird eine spätere Änderung der Entscheidungsgrundlage dadurch nicht automatisch erneut bewertet.

1.2 Wissenschaftliche Fragestellung

Diese Arbeit untersucht, inwieweit Attribute Stream-Based Access Control im Linux-Kernel prototypisch umgesetzt werden kann. Im Mittelpunkt steht dabei die Frage, wie dynamische Attribute aus dem Userspace für Zugriffsentscheidungen im Kernel bereitgestellt und bei Änderungen nachträglich berücksichtigt werden können.

Daraus ergibt sich folgende wissenschaftliche Fragestellung:

Wie kann Attribute Stream-Based Access Control mit dynamischen Attributen im Linux-Kernel prototypisch umgesetzt werden, und welche technischen Einschränkungen ergeben sich dabei?

Zur Beantwortung dieser Fragestellung werden insbesondere die Überführung textueller Policy-Dateien in eine kernelgeeignete Repräsentation, die Bereitstellung dynamischer Attribute für Zugriffsentscheidungen im Kernel sowie die nachträgliche Überwachung bestehender Zugriffssituationen betrachtet.

1.3 Zielsetzung

Zielsetzung der Arbeit ist ein Prototyp, anhand dessen eine Zugriffskontrolle mittels Attribute Stream-Based Access Control im Linux-Kernel stattfindet. Insbesondere sollen Stream-Attribute wie Zeitinformationen sowie externe, zur Laufzeit aktualisierte Attribute in die Zugriffsentscheidung einbezogen werden. Ziel ist die Implementierung eines Prototyps, der Policies einliest, in den Kernel überträgt und Zugriffsentscheidungen anhand dieser Policies trifft. Darüber hinaus soll der Prototyp bereits bestehende Zugriffssituationen bei nachträglich geänderten Bedingungen erkennen und einschränken können.

1.4 Aufbau der Arbeit

2 Grundlagen

Dieses Kapitel beschreibt die fachlichen und technischen Grundlagen, auf denen der in dieser Arbeit entwickelte Prototyp aufbaut. Zunächst werden grundlegende Konzepte der Zugriffskontrolle eingeordnet, insbesondere das Reference-Monitor-Konzept sowie attributbasierte Zugriffskontrollmodelle. Darauf aufbauend wird erläutert, wie Attribute in dynamischen Systemen als veränderliche Entscheidungsgrundlage verwendet werden können und welche Rolle textuelle Policy-Beschreibungen dabei einnehmen.

Für die Umsetzung im Linux-Kernel sind außerdem die Trennung zwischen Userspace und Kernel space, Linux Security Modules sowie eBPF relevant. Diese Mechanismen bestimmen, an welchen Stellen Zugriffsentscheidungen technisch getroffen werden können und welche Einschränkungen bei der Implementierung im Kernel gelten. Ein besonderer Fokus liegt dabei auf eBPF-Maps als Schnittstelle zwischen Userspace und Kernel space, gepinnten Maps für dauerhaft zugreifbare Zustände sowie den Vorgaben des eBPF-Verifiers.

Abschließend werden die für den Prototyp spezifischen Grundlagen betrachtet: die Verwendung von Aya als Rust-Framework, die Abbildung dynamischer Attribute und Policy-Generationen sowie die Überwachung bereits bestehender Zugriffe über File Descriptors. Damit schafft das Kapitel die Grundlage für die anschließende Anforderungsanalyse und die spätere Architektur des Systems.

2.1 Zugriffskontrolle und Reference-Monitor-Konzept

2.2 Attribute-Based Access Control

2.3 Attribute Stream-Based Access Control

2.4 Policy-Beschreibungen für ASBAC

2.5 Userspace und Kernel space in Linux

2.6 Linux Security Modules und LSM-Hooks

2.7 eBPF und eBPF-LSM

2.8 eBPF-Maps und gepinnte Maps

2.9 eBPF-Verifier, Stack und Programmkomplexität

2.10 Aya als Rust-Framework für eBPF

2.11 Dynamische Attribute und Policy-Generationen

2.12 Überwachung bestehender Zugriffe und File Descriptors

3 Anforderungsanalyse

In diesem Kapitel werden die Anforderungen an den zu entwickelnden Prototypen beschrieben. Die Anforderungen ergeben sich aus der Zielsetzung, zu untersuchen, wie Attribute Stream-Based Access Control im Linux-Kernel prototypisch umgesetzt werden kann. Sie beschreiben das Verhalten, das der Prototyp im Rahmen der Arbeit zeigen soll.

3.1 Funktionale Anforderungen

Funktionale Anforderungen beschreiben, welche fachlichen Funktionen der Prototyp bereitstellen muss. Sie sind im Regelfall direkt testbar, da geprüft werden kann, ob eine Funktion vorhanden ist und das erwartete Verhalten zeigt. Im Kontext dieser Arbeit betreffen sie insbesondere das Einlesen und Verarbeiten von Policies, die Zugriffskontrolle über LSM-Hooks sowie die nachträgliche Überwachung bestehender Zugriffe.

FA-01 Policy-Einlesen: Der Prototyp muss ASBAC-Policies aus einer definierten Quelle im Userspace einlesen können.

Akzeptanzkriterium: Eine im Policy-Verzeichnis abgelegte Policy wird erkannt und für die weitere Verarbeitung bereitgestellt.

FA-02 Policy-Verwaltung: Der Prototyp muss Änderungen an Policies erkennen und aktualisierte Policies in die Zugriffskontrolle übernehmen können.

Akzeptanzkriterium: Wird eine Policy geändert, hinzugefügt oder entfernt, wird diese Änderung vom System verarbeitet.

FA-04 Zugriffskontrolle über LSM-Hooks: Der Prototyp muss sicherheitsrelevante Operationen über Linux-Security-Module-Hooks abfangen können.

Akzeptanzkriterium: Ein relevanter Zugriff auf eine Ressource wird durch einen LSM-Hook kontrolliert.

FA-05 Entscheidungsfindung anhand von Policies: Der Prototyp muss Zugriffsentscheidungen auf Grundlage der geladenen Policies treffen können.

Akzeptanzkriterium: Ein Zugriff wird abhängig von einer Policy erlaubt oder verweigert.

FA-06 Unterstützung dynamischer Attribute: Der Prototyp muss dynamische Attribute, beispielsweise Zeitinformationen oder externe Zustandsinformationen, in Zugriffsentscheidungen einbeziehen können.

Akzeptanzkriterium: Eine Änderung eines dynamischen Attributs kann zu einer veränderten Zugriffsentscheidung führen.

FA-07 Überwachung bestehender Dateizugriffe: Der Prototyp muss bereits bestehende Dateizugriffe überwachen können, wenn sich entscheidungsrelevante Policies oder Attribute zur Laufzeit ändern. Als bestehender Zugriff gilt im Rahmen des Prototyps ein geöffneter File Descriptor eines laufenden Prozesses auf eine durch eine Policy erfasste Datei. Der Monitor muss erkennen können, welcher Prozess über welchen

File Descriptor auf welche Datei zugreift und ob dieser Zugriff nach der aktuellen Policy- und Attributlage weiterhin zulässig ist.

Akzeptanzkriterium: Öffnet ein Prozess eine Datei zunächst zulässig und wird der Zugriff anschließend durch eine Änderung eines entscheidungsrelevanten Attributs unzulässig, erkennt der Monitor diese Verletzung und kann Prozess-ID, File Descriptor und betroffenen Dateipfad ausgeben.

FA-08 Enforcement bei unzulässig gewordenen Dateizugriffen: Der Prototyp muss einen bestehenden Dateizugriff beenden können, wenn dieser aufgrund einer Policy- oder Attributänderung nicht mehr zulässig ist. Im Rahmen des Prototyps erfolgt das Enforcement für Dateizugriffe durch das Schließen des betroffenen File Descriptors des jeweiligen Prozesses. Dabei darf nur der File Descriptor geschlossen werden, der den unzulässig gewordenen Zugriff repräsentiert.

Akzeptanzkriterium: Besitzt ein Prozess mehrere geöffnete File Descriptors und wird nur einer dieser Zugriffe durch eine Attribut- oder Policyänderung unzulässig, schließt der Prototyp nur diesen betroffenen File Descriptor. Andere zulässige File Descriptors desselben Prozesses bleiben geöffnet.

FA-09 Administrationsschnittstelle: Der Prototyp soll eine einfache Administrationschnittstelle bereitstellen, über die zentrale Informationen zum Zustand des Prototyps lesend eingesehen werden können.

Akzeptanzkriterium: Ein Administrator kann zentrale Informationen zum Zustand des Prototyps abrufen, ohne über die Administrationsschnittstelle Policies, Attribute oder Kernelzustand zu verändern.

FA-10 Gültige Policy-Generationen: Der Prototyp muss sicherstellen, dass nur erfolgreich erzeugte und validierte Generationen von Policies für Zugriffsentscheidungen verwendet werden. Fehlerhafte, unvollständige oder nicht erfolgreich verarbeitete Policy-Generationen dürfen nicht als aktive Entscheidungsgrundlage gelten.

Akzeptanzkriterium: Eine neue Policy-Generation wird erst dann aktiviert, wenn sie vollständig verarbeitet und erfolgreich validiert wurde. Andernfalls bleibt die zuletzt gültige Policy-Generation aktiv.

FA-11 Eingabvalidierung: Der Prototyp muss Eingaben aus dem Userspace validieren, bevor diese im Kernel für Zugriffsentscheidungen verwendet werden.

Akzeptanzkriterium: Ungültige oder manipulierte Eingaben werden erkannt und nicht als Entscheidungsgrundlage im Kernel verwendet.

FA-12 beliebige Attributnamen: In Policies muss es für den Nutzer möglich sein, Attributnamen frei zu vergeben, ohne dass diese vorher im Prototyp fest definiert sein müssen. Zulässige Attributnamen dürfen nicht leer sein und bestehen aus ASCII-Buchstaben (A-Z, a-z), Ziffern (0-9), Unterstrich (_) und Bindestrich (-).

Akzeptanzkriterium: Eine Policy mit einem gültigen, zuvor nicht bekannten Attributnamen, beispielsweise `subject.position == engineer`, wird erfolgreich verarbeitet. Eine Policy mit einem Attributnamen außerhalb des zulässigen Zeichensatzes wird abgelehnt.

3.2 Operative Anforderungen

Operative Anforderungen beschreiben, wie gut der Prototyp seine Funktionen im Betrieb erfüllen soll. Sie beziehen sich auf Eigenschaften, die für Nutzung, Betrieb und Evaluation sichtbar sind, beispielsweise Stabilität, Beobachtbarkeit, Nachvollziehbarkeit, Performance und Reproduzierbarkeit.

OA-01 Stabilität: Der Prototyp darf die Stabilität des Linux-Kernels nicht unnötig gefährden.

Akzeptanzkriterium: Fehlerhafte oder ungültige Eingaben aus dem Userspace führen nicht zu einem Kernel-Absturz.

OA-02 Beobachtbarkeit und Nachvollziehbarkeit: Der Prototyp soll zentrale Zustände und Entscheidungen so sichtbar machen, dass Betrieb und Evaluation nachvollziehen können, welche Policies, Attribute oder Zustände eine Zugriffsentscheidung beeinflusst haben.

Akzeptanzkriterium: Für zentrale Zugriffsentscheidungen ist erkennbar, welche Policy oder welcher Zustand die Entscheidung beeinflusst hat.

OA-03 Performance: Die zusätzlichen Prüfungen durch den Prototyp sollen den kontrollierten Zugriff nicht unverhältnismäßig verlangsamen.

Akzeptanzkriterium: Die Auswirkungen auf die Ausführungszeit werden in der Evaluation betrachtet.

OA-04 Reproduzierbarkeit: Die Evaluation des Prototyps soll unter dokumentierten Bedingungen wiederholbar sein.

Akzeptanzkriterium: Testumgebung, Policies und Testszenarien werden so beschrieben, dass die Ergebnisse nachvollzogen werden können.

3.3 Entwicklungsbezogene Anforderungen

Entwicklungsbezogene Anforderungen beschreiben Eigenschaften, die vor allem für die Entwicklung und spätere Erweiterung des Prototyps relevant sind. Sie sind für Nutzer nicht immer unmittelbar sichtbar, beeinflussen aber, wie verständlich, änderbar und erweiterbar das System bleibt.

EA-01 Wartbarkeit: Die Implementierung soll modular aufgebaut sein, sodass Userspace-Komponenten, Kernel-Komponenten und Kommunikationsschnittstelle getrennt betrachtet werden können.

EA-02 Erweiterbarkeit: Der Prototyp soll so entworfen werden, dass weitere Attribute, Policies oder LSM-Hooks ergänzt werden können.

EA-03 Begrenzter Kernel-Anteil: Komplexe Policy-Logik soll möglichst im Userspace verbleiben, um die Komplexität im Kernel zu reduzieren.

4 Konzeption

4.1 praktische Einschränkungen von eBPF

Ableitung daraus, was Policies genau können und was nicht.

4.2 Architektur

Die Architektur des Prototyps ist in zwei zentrale Bereiche unterteilt: den Userspace und den Kernelspace. Diese Trennung orientiert sich an der grundsätzlichen Struktur von Linux-Systemen und ermöglicht es, komplexe Logik zur Policy-Verarbeitung außerhalb des Kernels auszuführen, während sicherheitskritische Zugriffentscheidungen direkt im Kernel umgesetzt werden.

- **Userspace:** Im Userspace befinden sich die Komponenten zur Verwaltung und Verarbeitung von Policies. Dieser unterteilt sich in das Hauptprogramm, einen Attributloader, einen Monitor und der Policyverwaltung. Das Hauptprogramm lädt das Kernelprogramm und startet dieses. Der Attributloader lädt Streamingattribute in den Kernelspace. Die Policyverwaltung liest die textuellen Policy-Dateien, parst diese und übersetzt sie in kernelgeeignete Map-Einträge. Der Monitor überwacht Dateizugriffe im Userspace und prüft mögliche Policy-Verletzungen. Durch eine nachträgliche Durchsetzungsmaßnahme kann der Prozess bei einer Policy-Verletzung beendet oder der Dateizugriff durch das Schließen des File Descriptors beendet werden.
- **Kernelspace:** Im Kernelspace wird ein Linux Security Module integriert. Dieses greift über LSM-Hooks in sicherheitsrelevante Operationen ein und trifft Zugriffsentscheidungen auf Basis der vom Userspace bereitgestellten Policies. Hierbei wertet das LSM auch Streamingattribute, welche vom Userspace aus in den Kernel gestreamt werden aus.

Die Kommunikation zwischen Userspace und Kernelspace bildet dabei eine zentrale Schnittstelle des Systems. Sie dient dazu, Policies, Entscheidungen oder Attribute zwischen beiden Bereichen auszutauschen. Dadurch kann die Policy-Logik im Userspace verbleiben, während der Kernel die unmittelbare Durchsetzung der Zugriffskontrolle bei Dateizugriffen übernimmt. Für Zugriffe, die bereits durch einen LSM-Hook erlaubt wurden, erfolgt keine automatische erneute Bewertung durch denselben Hook. Diese nachträgliche Überwachung wird durch den Monitor übernommen.

4.3 Datenfluss

Der Datenfluss des Prototyps lässt sich in vier Teilbereiche unterteilen. Diese Teilbereiche beschreiben, wie Policies in das System gelangen, wie dynamische Attribute aktualisiert werden, wie ein Zugriff im Kernel bewertet wird und wie bereits bestehende Zugriffe nachträglich überwacht werden.

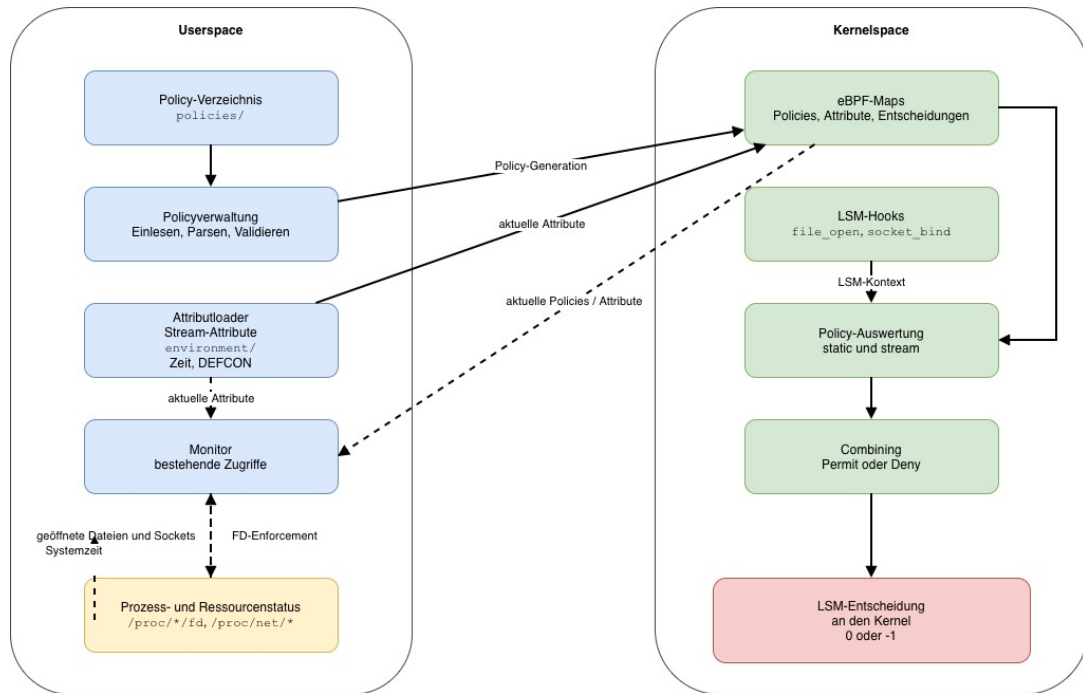


Abbildung 1: Konzeptionelle Architektur des Prototyps

4.3.1 Policy-Datenfluss

Policies werden im Userspace in einem dafür vorgesehenen Policy-Verzeichnis abgelegt. Die Policyverwaltung überwacht dieses Verzeichnis und erkennt, wenn Policies hinzugefügt, geändert oder entfernt werden. Nach einer Änderung werden die vorhandenen Policy-Dateien eingelesen und in kernelgeeignete Map-Einträge übersetzt. Dabei wird geprüft, ob die Policies syntaktisch und semantisch für den Prototyp verarbeitbar sind.

Erst wenn diese Verarbeitung erfolgreich abgeschlossen ist, werden die Policies als neue Policy-Generation in den Kernel übertragen. Die Übertragung erfolgt über gepinnte eBPF-Maps, die als gemeinsame Datenstruktur zwischen Userspace und Kernel space dienen. Durch das Generationenmodell wird verhindert, dass der Kernel während einer Zugriffsentscheidung eine nur teilweise aktualisierte Policy-Menge verwendet. Schlägt die Verarbeitung oder Übertragung fehl, bleibt die zuletzt gültige Policy-Generation aktiv.

4.3.2 Datenfluss dynamischer Attribute

Neben den Policies werden dynamische Attribute aus dem Userspace in den Kernel space übertragen. Dynamische Attribute sind Werte, die sich zur Laufzeit ändern und deshalb regelmäßig aktualisiert werden müssen. Im Prototyp betrifft dies insbesondere Zeitinformationen sowie vpm Nutzer frei wählbare Attribute (vgl. FA-12).

Die Userspace-Komponente ermittelt diese Werte beziehungsweise liest sie aus einer definierten Quelle ein. Anschließend werden die aktuellen Werte in dafür vorgesehene eBPF-Maps geschrieben. Der Kernel kann diese Werte bei der Auswertung von Policies lesen und in die Zugriffsentscheidung einbeziehen.

4.3.3 Zugriffsentscheidungen im Kernel

Wenn ein kontrollierter Zugriff stattfindet, wird im Kernel ein LSM-Hook ausgelöst. Bei einem Dateizugriff ist dies beispielsweise der Hook `file_open`. Das eBPF-Programm liest aus dem Hook-Kontext die für die Entscheidung relevanten Informationen aus. Dazu gehören unter anderem das Subjekt, die angefragte Ressource und der aktuelle Prozesskontext.

Auf Grundlage dieser Zugriffsdaten werden die zugehörigen Policies im Kernel ausgewertet. Dabei werden nur Policies berücksichtigt, die für den jeweiligen Hook vorgesehen und auf die konkrete Zugriffssituation anwendbar sind. Eine Policy ist beispielsweise nur dann anwendbar, wenn Benutzer, Prozessname und Ressource zu der aktuellen Anfrage passen. Erst wenn eine Policy anwendbar ist, kann sie eine Aussage in Form von *Permit* oder *Deny* treffen.

Die Zwischenergebnisse der Policy-Auswertung werden anschließend zusammengeführt. Das Combining entscheidet, ob der Zugriff insgesamt erlaubt oder verweigert wird. Führt das Ergebnis zu einer Verweigerung, gibt das eBPF-LSM-Programm einen Fehlerwert an den Kernel zurück. Der Kernel unterbindet daraufhin den Zugriff.

4.3.4 Monitoring bestehender Zugriffe

LSM-Hooks prüfen einen Zugriff zu dem Zeitpunkt, an dem das jeweilige Kernelereignis auftritt. Bei streambasierten Policies kann sich die Entscheidungslage jedoch nachträglich ändern, beispielsweise wenn sich die Uhrzeit oder andere Attribute ändern. Ein bereits geöffneter Zugriff wird dadurch nicht automatisch erneut durch denselben LSM-Hook geprüft.

Aus diesem Grund enthält der Prototyp zusätzlich einen Monitor im Userspace. Der Monitor betrachtet regelmäßig aktive Prozesse und deren aktuell verwendete Ressourcen. Dabei prüft er, ob eine bestehende Zugriffssituation mit den derzeit aktiven Policies vereinbar ist. Wird eine Verletzung erkannt, kann der Monitor ein nachträgliches Enforcement auslösen. Im Fall von Dateizugriffen kann dies beispielsweise bedeuten, dass betroffene File Descriptors eines Prozesses geschlossen werden.

Der Monitor ergänzt damit die kernelbasierte Zugriffskontrolle. Während die LSM-Hooks neue Zugriffe unmittelbar prüfen, adressiert der Monitor das Problem bereits bestehender Zugriffe bei nachträglich geänderten Bedingungen.

4.4 Komponenten

Die Architektur des Prototyps setzt sich aus mehreren Komponenten zusammen, die jeweils eine abgegrenzte Aufgabe innerhalb der Zugriffskontrolle übernehmen. Im Folgenden werden die wichtigsten Komponenten beschrieben.

4.4.1 Userspace-Komponenten

Im Userspace befinden sich die Komponenten, die Policies verwalten, dynamische Attribute bereitstellen und bestehende Zugriffssituationen überwachen. Dadurch verbleiben komplexere Verarbeitungsschritte außerhalb des Kernels.

Policies Policies beschreiben die Zugriffsvorgaben des Systems. Sie legen fest, welches Subjekt unter welchen Bedingungen auf welche Ressource zugreifen darf oder nicht zugreifen darf. Im Prototyp liegen sie als Textdateien im Policy-Verzeichnis `policies/`. Policies sind damit keine ausführbare Programmlogik, sondern Eingabedaten für die Policyverwaltung und die spätere Auswertung im Kernelspace.

Policyverwaltung Die Policyverwaltung bildet die Schnittstelle zwischen den menschenlesbaren Policy-Dateien und den im Kernel verwendbaren Map-Einträgen. Sie überwacht das Verzeichnis `policies/`, liest die dort abgelegten Policies ein, verarbeitet sie und validiert sie vor der Aktivierung. Nach erfolgreicher Validierung übersetzt der Userspace die Policy-Dateien in kernelgeeignete Map-Einträge.

Da Policies während des Betriebs geändert werden können, muss verhindert werden, dass der Kernel eine nur teilweise aktualisierte Policy-Menge verwendet. Dazu aktiviert die Policyverwaltung Policies als atomare Einheit. Dies wird durch ein Generationensystem gewährleistet: Eine neue Policy-Generation wird zunächst vollständig im Userspace vorbereitet und validiert. Erst danach wird sie in den Kernelspace übertragen und als gültige Generation aktiviert. Schlägt dieser Vorgang fehl, bleibt die zuletzt gültige Generation aktiv.

Die Policyverwaltung umfasst damit gelesene Policy-Dokumente, geparste Policies, kernelgeeignete Map-Einträge, Policy-Generationen sowie aktive und zuletzt gültige Policy-Versionen.

Attributloader Der Attributloader stellt dynamische Kontextinformationen für die Zugriffskontrolle bereit. Diese Attribute sind nicht notwendigerweise Bestandteil eines einzelnen Zugriffsvorgangs, können aber die Entscheidung beeinflussen. Im Prototyp betrifft dies insbesondere die aktuelle Zeit sowie externe Attribute aus dem Verzeichnis `environment/`. Der Attributloader schreibt diese Werte in den Kernelspace, damit sie dort bei der Auswertung von Streampolicies berücksichtigt werden können.

Der Attributloader trifft selbst keine Zugriffsentscheidung. Seine Aufgabe besteht darin, die Entscheidungsgrundlage aktuell zu halten.

Monitor Der Monitor ergänzt die kernelbasierte Zugriffskontrolle um die nachträgliche Überwachung bereits bestehender Zugriffssituationen. Dies ist notwendig, weil ein LSM-Hook einen Zugriff nur zum Zeitpunkt des jeweiligen Kernelereignisses prüft. Ändern sich danach dynamische Attribute, wird ein bereits bestehender Zugriff nicht automatisch erneut durch denselben Hook geprüft.

Der Monitor betrachtet deshalb regelmäßig laufende Prozesse und deren verwendete Ressourcen. Dazu nutzt er den Prozess- und Ressourcenstatus des Betriebssystems, beispielsweise Informationen über offene File Descriptors. Diese Informationen werden gegen die aktuell aktiven Policies geprüft. Wird eine Verletzung erkannt, kann ein nachträgliches Enforcement ausgelöst werden. Bei Dateizugriffen kann dies beispielsweise bedeuten, dass ein betroffener File Descriptor geschlossen und damit der Zugriff auf eine bereits geöffnete Ressource beendet wird.

Der Monitor ersetzt damit nicht die Zugriffskontrolle im Kernelspace, sondern ergänzt sie für Zugriffssituationen, die nach der ursprünglichen Hook-Entscheidung unzulässig werden.

4.4.2 Kernelspace-Komponenten

LSM Die LSM-Komponente bildet den kernelbasierten Durchsetzungspunkt des Prototyps. Sie greift über LSM-Hooks in sicherheitsrelevante Operationen ein und erhält dadurch Zugriff auf die für die Entscheidung relevanten Zugriffsdaten. Auf dieser Grundlage werden die im Kernelspace vorliegenden Policies ausgewertet. Das Ergebnis dieser Auswertung entscheidet, ob der jeweilige Zugriff erlaubt oder verweigert wird.

4.4.3 Administrationskomponente

Administrationstool Das Administrationstool dient der Diagnose des Prototyps. Es ermöglicht, zentrale Informationen zum Zustand des Systems, über geladene Policies und Attribute lesend einzusehen. Zur Gewährleistung der Konsistenz des Systems wird hier auf eine schreibende Funktion verzichtet.

4.5 Datenobjekte

Neben den aktiven Komponenten besitzt der Prototyp mehrere zentrale Datenobjekte. Diese Datenobjekte beschreiben den Zustand des Systems und bilden die Grundlage für Policy-Verarbeitung, Zugriffsauswertung und Monitoring. Im Folgenden wird zwischen Datenobjekten im Userspace und Datenobjekten im Kernelspace unterschieden.

4.5.1 Datenobjekte im Userspace

Im Userspace liegen die Datenobjekte, die für die Verwaltung, Vorbereitung und nachträgliche Überwachung von Policies benötigt werden. Dazu gehören zunächst die Policy-Dateien im Verzeichnis `policies/`. Sie stellen die menschenlesbare Beschreibung der Zugriffsvorgaben dar.

Während der Verarbeitung entstehen daraus weitere Datenobjekte: gelesene Policy-Dokumente, geparste Policies und kernelgeeignete Map-Einträge. Diese Zwischenformen sind notwendig, um die textuellen Policies in eine strukturierte Form zu überführen, die später in eBPF-Maps im Kernelspace übertragen werden kann. Zusätzlich verwaltet der Userspace Policy-Generationen. Eine Policy-Generation beschreibt eine zusammengehörige Menge von Policies, die gemeinsam aktiviert oder verworfen wird.

Weitere Datenobjekte im Userspace sind dynamische Attribute wie Zeitinformationen oder externe Zustandswerte aus dem Verzeichnis `environment/`. Diese Werte werden vom Attributloader aktualisiert und in den Kernelspace übertragen.

Für den Monitor sind außerdem Prozess- und Ressourceninformationen relevant. Dazu zählen laufende Prozesse, Benutzer-IDs, Prozessnamen, offene File Descriptors, geöffnete Dateien und aktive Sockets. Diese Informationen stammen aus Betriebssystemquellen wie `/proc/<PID>/fd` und werden verwendet, um bestehende Zugriffssituationen gegen die aktiven Policies zu prüfen.

4.5.2 Datenobjekte im Kernelspace

Im Kernelspace liegen die Datenobjekte, die für die unmittelbare Zugriffskontrolle an den LSM-Hooks benötigt werden. Dazu gehören die geladenen statischen Policies und

Streampolicies. Sie werden nach erfolgreicher Validierung aus dem Userspace übertragen und in eBPF-Maps abgelegt.

Neben den Policies befinden sich auch dynamische Attribute im Kernelspace. Dazu zählen insbesondere die aktuelle Zeit und externe Zustandswerte, die vom Attributloader bereitgestellt werden. Diese Attribute dienen der Auswertung von Streampolicies.

Bei einem Zugriff erzeugt der Kernelspace außerdem Zugriffsdaten. Diese beschreiben die konkrete Anfrage, beispielsweise das Subjekt, den Prozessnamen, die betroffene Ressource oder Socket-Informationen. Auf Grundlage dieser Zugriffsdaten werden die geladenen Policies ausgewertet.

Schließlich existieren im Kernelspace Zwischenergebnisse der Entscheidungsfindung. Sie halten fest, ob während der Auswertung eine anwendbare Permit- oder Deny-Policy gefunden wurde. Diese Zwischenergebnisse werden im Combining zusammengeführt und bestimmen die finale Entscheidung des LSM-Hooks.

4.6 Entwurfsentscheidungen

4.6.1 Trennung von Userspace und Kernelspace

Das Reference-Monitor-Konzept bildet eine grundlegende theoretische Basis für die Umsetzung von Zugriffskontrollmechanismen. Nach Anderson muss ein Reference Monitor manipulationssicher und verifizierbar sein. Weiterhin muss eine vollständige Überprüfung aller Zugriffe auf geschützte Ressourcen gewährleistet sein [1]. Wright et al. beschreiben, dass das LSM-Framework Sicherheits-Hooks an sicherheitskritischen Stellen des Kernels einfügt und dass diese Hooks Rückgabewerte liefern, welche über Erfolg oder Fehler der angeforderten Operation entscheiden. Daraus folgt, dass die Zugriffskontrollentscheidung innerhalb des Kernel-Ausführungspfades erfolgen muss [6].

Daraus folgt jedoch nicht, dass die gesamte Policy-Verarbeitung im Kernelspace erfolgen muss. Insbesondere das Einlesen, Parsen und Validieren textueller Policy-Dateien ist vergleichsweise komplex und fehleranfällig. Diese Verarbeitungsschritte eignen sich daher besser für den Userspace, da Fehler dort nicht unmittelbar die Stabilität des Kernels gefährden. Der Kernelspace wird im Prototyp auf die sicherheitskritische Auswertung einer bereits vorbereiteten Policy-Repräsentation beschränkt.

Die gewählte Architektur trennt daher zwischen Policy-Verarbeitung im Userspace und Zugriffsdurchsetzung im Kernelspace. Der Userspace übernimmt die Verwaltung der Policies, die Bereitstellung dynamischer Attribute und die Überwachung bereits bestehender Zugriffssituationen. Der Kernelspace übernimmt die unmittelbare Entscheidung an den LSM-Hooks. Damit wird die Komplexität im Kernel reduziert, während die Zugriffskontrolle weiterhin im sicherheitskritischen Ausführungspfad des Kernels durchgesetzt wird.

4.6.2 Einsatz von eBPF-LSM statt klassischem Kernelmodul

Ein klassisches LSM ist Teil des Kernelcodes oder wird als Kernelmodul entwickelt. Änderungen an einem solchen Modul erfordern einen erneuten Build- und Ladeprozess. Für einen experimentellen Prototyp führt dies zu vergleichsweise langen Entwicklungsiterationen.

Gewählt wurde daher die Implementierung als eBPF-LSM. Diese Implementierung erlaubt es, eBPF-Programme an LSM-Hooks auszuführen und damit Zugriffskontrollentscheidungen im Kernel-Ausführungspfad zu treffen [4]. Die Programme können zur Laufzeit geladen werden und werden vor der Ausführung durch den eBPF-Verifier geprüft [5].

Ein weiterer Grund für diese Entscheidung ist die Möglichkeit, eBPF-Programme mit Rust zu entwickeln. Der Prototyp nutzt dafür Aya. Aya ist eine Rust-Bibliothek für die Entwicklung von eBPF-Programmen und stellt damit eine Alternative zu einer C-basierten Implementierung dar [2]. Dadurch können Userspace- und eBPF-Anteile innerhalb desselben Rust-Workspaces entwickelt werden. Zusätzlich können gemeinsame Datenstrukturen und Teile der Policy-Auswertungslogik in einem gemeinsamen Crate abgebildet werden. Dies verbessert die Konsistenz zwischen Userspace und Kernelspace und reduziert doppelte Implementierungen.

Eine Alternative wäre die Entwicklung eines klassischen LSM gewesen. Diese Alternative bietet grundsätzlich größere Freiheiten im Kernelcode, erhöht jedoch Entwicklungs- und Wartungsaufwand. Für den Prototyp ist eBPF-LSM geeigneter, da Änderungen schneller getestet werden können und gleichzeitig ein sicherheitsrelevanter Ausführungspunkt im Kernel genutzt wird.

Konsequenzen dieser Entscheidung sind, dass die Implementierung die Einschränkungen von eBPF beachten muss, insbesondere die Prüfung durch den Verifier, die begrenzte Stack-Nutzung und die eingeschränkte Programmkomplexität. Diese Einschränkungen beeinflussen unter anderem die Aufteilung der eBPF-Programme in verschiedene Tail-Calls und die reduzierte Policy-Repräsentation im Kernelspace.

4.6.3 Auswahl der LSM-Hooks

4.6.4 Policy-Verarbeitung im Userspace

4.6.5 Reduzierte Policy-Repräsentation im Kernelspace

4.6.6 Kommunikation über eBPF-Maps

4.6.7 Generationenmodell für Policy-Aktualisierungen

4.6.8 Trennung statischer Policies und Streampolicies

4.6.9 Aufteilung der Policy-Maps nach Hook und Policy-Typ

4.6.10 Tail-Call-basierte Aufteilung der eBPF-Programme

4.6.11 Combining-Strategie für Policy-Entscheidungen

4.6.12 Identifikation von Dateien über Device und Inode

4.6.13 Nachträgliche Überwachung durch den Monitor

4.6.14 Enforcement durch Schließen von File Descriptors

4.6.15 Administration und Diagnose über ein separates Tool

5 Entwurf und Umsetzung

5.1 Komponente 1

5.2 Komponente 2

5.3 Komponente 3

5.4 Implementierungsdetails

6 Evaluation

6.1 Testumgebung

6.2 Testszenarien

6.3 Ergebnisse

7 Zusammenfassung und Ausblick

Literatur

- [1] James P Anderson. Computer security technology planning study. Technical report, 1972.
- [2] The Aya Contributors. Building eBPF programs with aya. <https://aya-rs.dev/book/>, 2026. Zugriff am 08.06.2026.
- [3] The Kernel Development Community. Linux security module usage, 2025. URL <https://docs.kernel.org/admin-guide/LSM/index.html>.
- [4] The Kernel Development Community. Lsm bpf programs. https://docs.kernel.org/bpf/prog_lsm.html, 2026. Zugriff am 08.06.2026.
- [5] The Kernel Development Community. eBPF verifier. <https://docs.kernel.org/bpf/verifier.html>, 2026. Zugriff am 08.06.2026.
- [6] Chris Wright, Crispin Cowan, Stephen Smalley, James Morris, and Greg Kroah-Hartman. Linux security modules: General security support for the linux kernel. In *11th USENIX security symposium (USENIX Security 02)*, 2002.